BANKING MANAGEMENT SYSTEM

Software Design Description

Architecture, modules, data model, protocol, and synchronization

CS-513 • System Software Mini Project

Source baseline: repository main branch + supplied source archive

Prepared 25 October 2026 | Version 1.0

https://github.com/Dev2907/Banking-Management-IIITB

SYSTEM DESIGN

1. Design Overview

The system follows a client/server model with a terminal client, a TCP server, role-specific business-logic classes, and SQLite persistence. The server is authoritative: the client requests operations and displays responses; the server authenticates and invokes the appropriate role portal.

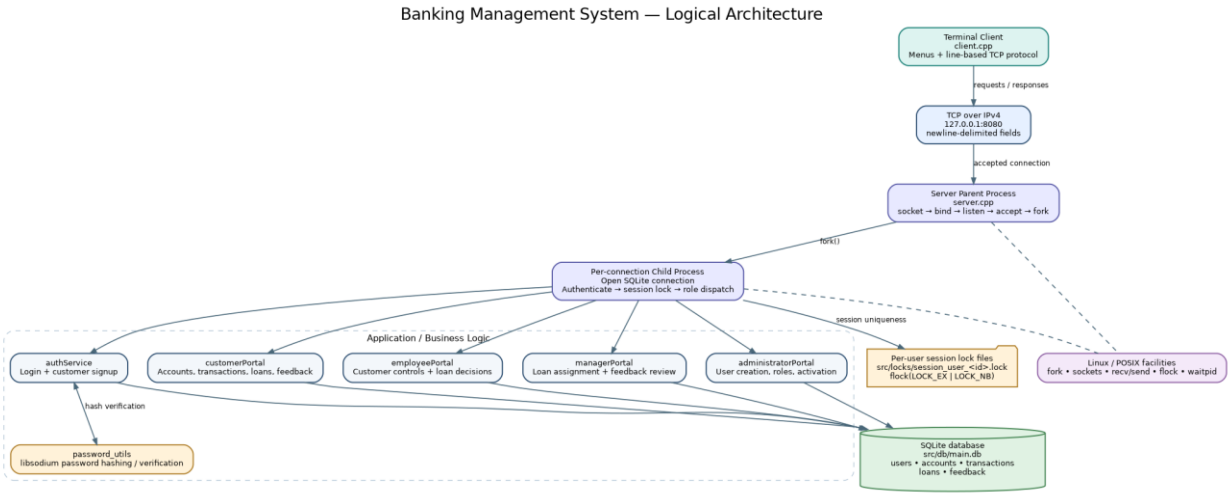


Figure 1. Logical architecture.

2. Architectural Elements

Component	Responsibility	Source
Terminal client	Connect, display role menus, send newline-delimited fields, parse responses.	src/client/client.cpp
Server parent	Create socket, bind, listen, accept connections, fork per client.	src/server/server.cpp
Client handler/session dispatch	Open SQLite connection in child, read login fields, authenticate, acquire session lock, dispatch by role.	src/server/server.cpp
authService	Authenticate username/password/role; register customer and create zero-balance account.	src/server/auth_service.cpp
customerPortal	Account selection and customer transactions, loan request, feedback, password change.	src/server/customer_portal.cpp
employeePortal	Customer username/active state, assigned loan list, loan decision, customer history, overview, password change.	src/server/employee_portal.cpp
managerPortal	Customer username/active state, loan assignment/listing, overview, feedback listing, password change.	src/server/manager_portal.cpp
administratorPortal	Create a user, modify user username/role/active state, password change.	src/server/administrator_portal.cpp
password_utils	libsodium hash and verification wrapper.	src/server/password_utils.cpp
SQLite database	Persist users, accounts, transactions, loans, feedback.	src/db/create_db_schema.sql
Session lock	Exclusive nonblocking per-user process-level session lock.	sessionLock in server.cpp

3. Module / Class Diagram

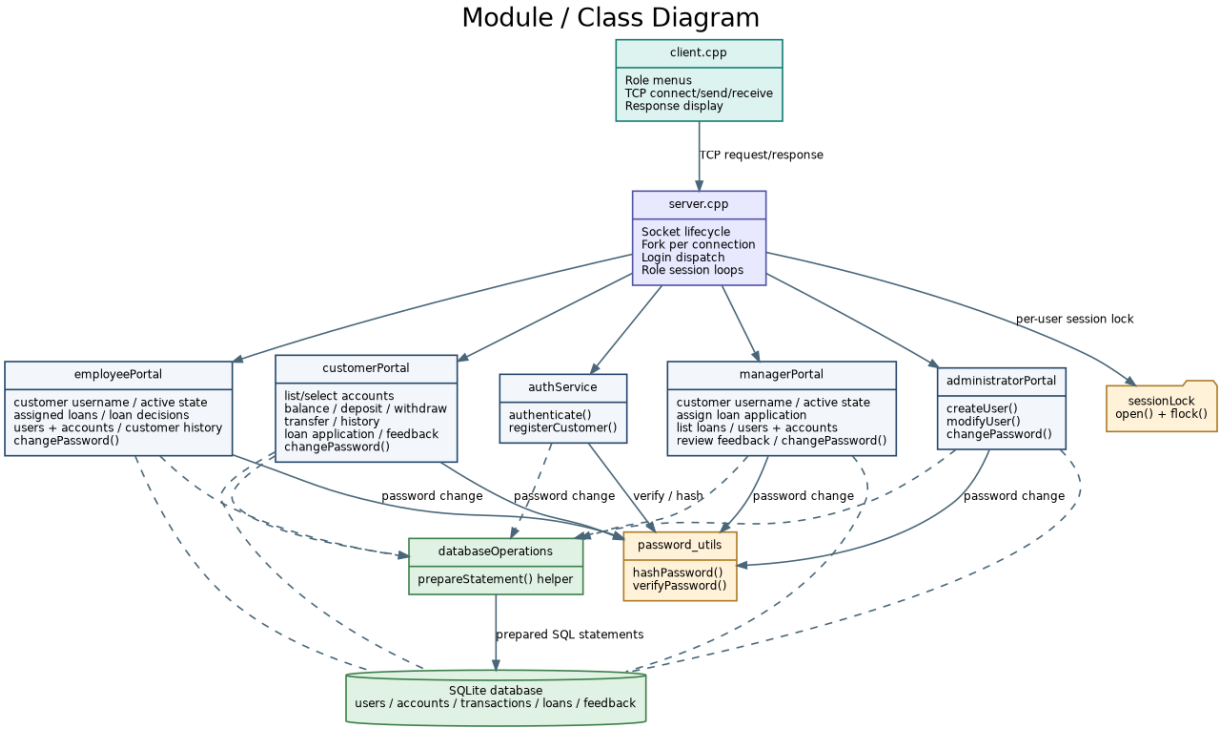


Figure 2. Logical module/class relationships.

4. Process and Session Lifecycle

1. The server creates an IPv4 TCP socket and binds to 127.0.0.1:8080.
2. The parent process listens and accepts a client connection.
3. The parent forks a child process; the parent closes its copy of the accepted socket, and the child closes its copy of the listening socket.
4. The child opens its own SQLite connection, reads the initial operation, role, username, and password fields, and calls authService.
5. On successful authentication, the child attempts to take an exclusive, nonblocking flock on src/locks/session_user_<userID>.lock.
6. If the lock succeeds, the child sends the login response and runs the role-specific session loop until LOGOUT or disconnect.
7. The child process termination closes process-held descriptors. The parent attempts waitpid(..., WNOHANG) immediately for the just-forked child; robust asynchronous child reaping is a known improvement area.

5. Data Model / ER Diagram

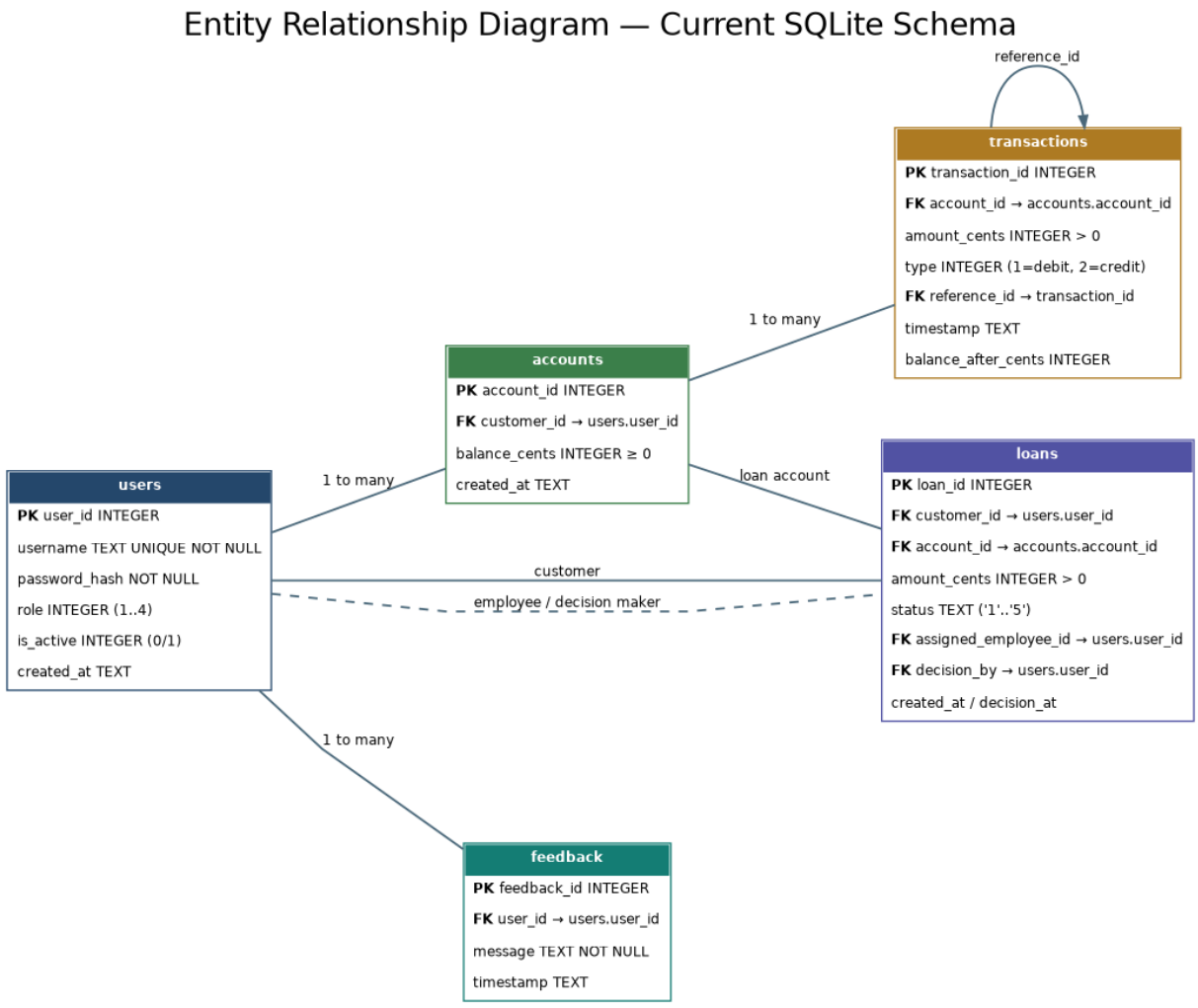


Figure 3. ER diagram generated from src/db/create_db_schema.sql.

5.1 Table Dictionary

Table	Fields and constraints	Meaning
users	user_id PK; username UNIQUE NOT NULL; password_hash NOT NULL; role 1..4; is_active 0/1; created_at	Identity, authentication, role and login state.
accounts	account_id PK; customer_id FK users; balance_cents ≥ 0; created_at	One customer can be associated with multiple account rows.
transactions	transaction_id PK; account_id FK; amount_cents > 0; type 1/2; reference_id self-FK; timestamp; balance_after_cents > 0	Ledger entries. In current handlers, type 1 is debit and type 2 is credit.
loans	loan_id PK; customer_id FK; account_id FK; amount_cents > 0; status 1..5; assigned_employee_id FK; decision_by FK; created_at; decision_at	Loan request and workflow metadata.
feedback	feedback_id PK; user_id FK; message NOT NULL; timestamp	Feedback submitted by a user; manager query filters customers.

5.2 Enumerations and Currency

Code	Interpretation in current handlers
User role 1	Customer
User role 2	Employee
User role 3	Manager
User role 4	Administrator
Transaction type 1	Debit (withdrawal and source side of transfer)
Transaction type 2	Credit (deposit, recipient side of transfer, approved loan disbursement)
Loan status '1'	Pending / ready for manager assignment
Loan status '2'	Assigned / awaiting employee decision
Loan status '3'	Approved; approval path credits the account
Loan status '4'	Rejected
Loan status '5'	Allowed by schema; not set by current handlers
Amounts	Integer smallest currency units; code does not parse decimal-point money strings.

6. Network Protocol Design

The client and server use a simple text protocol over a persistent TCP connection. recvLine() reads until a line-feed delimiter; sendLine() appends a line feed and sendAll() loops over partial writes. Each logical field occupies one line, so fields cannot safely contain embedded newlines.

6.1 Initial request

Line	Field	Example / constraints
1	operation	login or signup
2	role	1 customer, 2 employee, 3 manager, 4 administrator
3	username	Up to 64 characters expected by auth logic
4	password	Up to 1024 characters expected by auth logic

Signup is accepted only for role 1 (Customer). The authenticated role and user ID returned by authService determine which server-side session function is called.

6.2 Authenticated operation names

Session	Operations observed in server.cpp
Customer	GET_BALANCE; ADD_ACCOUNT; DEPOSIT + amount; WITHDRAW + amount; TRANSFER + recipient username + amount; APPLY_LOAN + amount; TRANSACTION_HISTORY; CHANGE_PASSWORD + current + new; FEEDBACK + message; LOGOUT.
Employee	EMP_CHANGE_USERNAME + current/new usernames; EMP_SET_ACTIVE + username/0-or-1; EMP_ASSIGNED_LOANS; EMP_VIEW_USERS; EMP_PROCESS_LOAN + loan ID + decision 3-or-4; EMP_CUSTOMER_HISTORY + username; EMP_CHANGE_PASSWORD; LOGOUT.

Session	Operations observed in server.cpp
Manager	MGR_CHANGE_CUSTOMER_USERNAME + current/new usernames; MGR_SET_USER_ACTIVE + username/0-or-1; MGR_ASSIGN_LOAN + employee username + loan ID; MGR_VIEW_LOANS; MGR_VIEW_USERS; MGR_VIEW_FEEDBACK; MGR_CHANGE_PASSWORD; LOGOUT.
Administrator	ADMIN_CREATE_USER + username + password + role 1..3; ADMIN_MODIFY_USER + current/new username + role 1..4 + active 0/1; ADMIN_CHANGE_PASSWORD; LOGOUT.

6.3 Response conventions

- Simple portal operations usually return a status line (1 success / 0 failure) followed by a human-readable message line or lines.
- List operations return a status line, a line count, and exactly that many message lines (with a special zero-count message path).
- Transaction history has its own status plus count framing; account selection sends a count and pairs of account ID and balance values.
- The current auth response construction depends on newline characters in the auth message to separate it from user ID and role. Standardizing all responses to an explicit count or length field is recommended.

7. Transaction Design and ACID

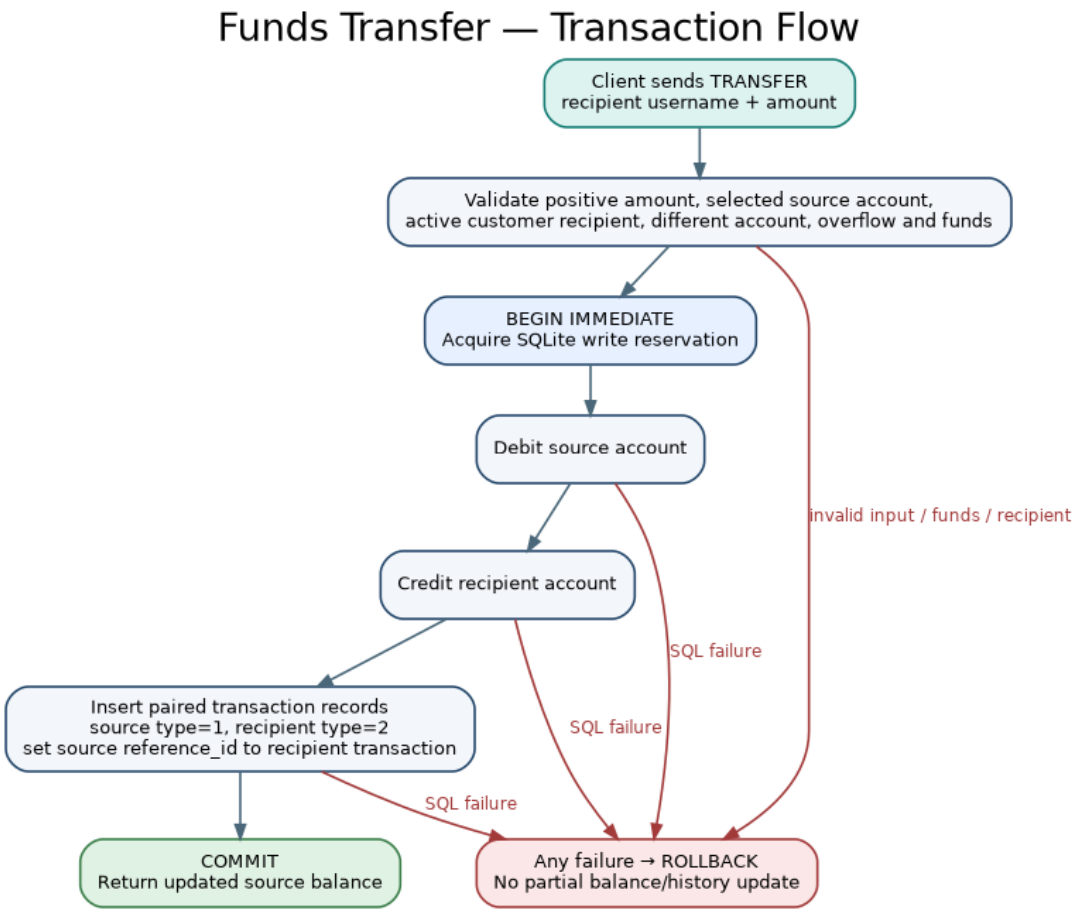


Figure 4. Transfer processing flow.

Property	Implementation mechanism	Evidence still needed

Property	Implementation mechanism	Evidence still needed
Atomicity	Transfer debits source, credits destination, writes paired ledger rows and reference ID within one transaction; other financial paths use transaction helpers or BEGIN IMMEDIATE.	Inject/force failures after intermediate steps and verify all changes roll back.
Consistency	Positive amount checks, non-negative balance checks in schema/SQL, sufficient-fund checks, role/ownership predicates.	Negative, zero, overflow, invalid target, and foreign-key tests.
Isolation	SQLite write transaction begins with BEGIN IMMEDIATE for important multi-step flows; database serializes writes.	Run multi-client deposit/withdraw/transfer stress tests.
Durability	Successful code paths commit to SQLite.	Restart server and verify balances/history remain unchanged after commit.

8. Security Controls

- Password hashing and verification use libsodium crypto_pwhash_str / crypto_pwhash_str_verify.
- Parameterized SQL statements are used for most values supplied by clients.
- The server uses authenticated IDs in portal constructors; employee/manager loan operations also check ownership/assignment in SQL predicates.
- Per-user lock paths are created with restrictive mode 0600 and opened with flock(LOCK_EX | LOCK_NB).
- The server listens on loopback only and the application has no TLS; do not expose this binary to an untrusted network.